

3 - Uninformed Search - BFS and DFS

Artificial Intelligence Lab Manual - Clean Study Edition

Learning Objectives

- Represent a search problem using states, actions, a start state, and a goal test.
- Implement Breadth-First Search and Depth-First Search.
- Compare completeness, memory use, and path quality.

Background

Uninformed search uses only the problem definition and does not estimate which state is closer to the goal. BFS is useful when step costs are equal and a shortest-step solution is required. DFS uses less memory but can follow an unproductive branch for a long time.

Core Concepts

Concept	Meaning in this lab
State space	All states reachable through valid actions.
Frontier	Generated states waiting to be expanded.
BFS	Expands the shallowest frontier nodes first.
DFS	Expands the deepest frontier node first.
Visited set	Prevents repeated expansion in graph search.

Guided Procedure

1. Represent a graph as an adjacency dictionary.
2. Implement BFS with a queue and a visited set. Store parent pointers so the path can be reconstructed.
3. Implement DFS with a stack or recursion and the same visited logic.
4. Run both algorithms on the same start and goal pair.
5. Record expansion order, path length, and maximum frontier size.

Reference Implementation

Use the following as a starting point. Read and modify it rather than treating it as a final answer.

```
from collections import deque

def bfs(graph, start, goal):
    q = deque([(start, [start])])
    seen = {start}
    while q:
        node, path = q.popleft()
        if node == goal:
            return path
        for nxt in graph.get(node, []):
            if nxt not in seen:
                seen.add(nxt)
                q.append((nxt, path + [nxt]))
    return None
```

Lab Exercises

Task 1. Implement DFS iteratively and return the first path found.

Task 2. Create a graph where BFS and DFS return different valid paths. Explain why.

Task 3. Count how many nodes each algorithm expands on a balanced tree of depth 4.

Task 4. Extension: add depth-limited DFS and identify a case where the depth limit matters.

Suggested Deliverables

- Notebook or Python file containing the completed implementation.
- Short result table or output evidence for each required comparison.
- A concise interpretation of what changed and why; do not submit output without explanation.

Review Questions

1. Why is BFS optimal only under particular cost assumptions?
2. Why can DFS use less memory?
3. What problem does a visited set solve?

Completion check: You should be able to explain the algorithm or workflow in your own words, reproduce the main result, and identify at least one limitation.